9 - Stacks and Queues

Abstract data types and a single abstraction layer are not the only level of abstraction we can introduce. We can add a variety of additional abstractions to divorce concepts from implementations including one or more policies to dictate specific behaviors for a data structure.

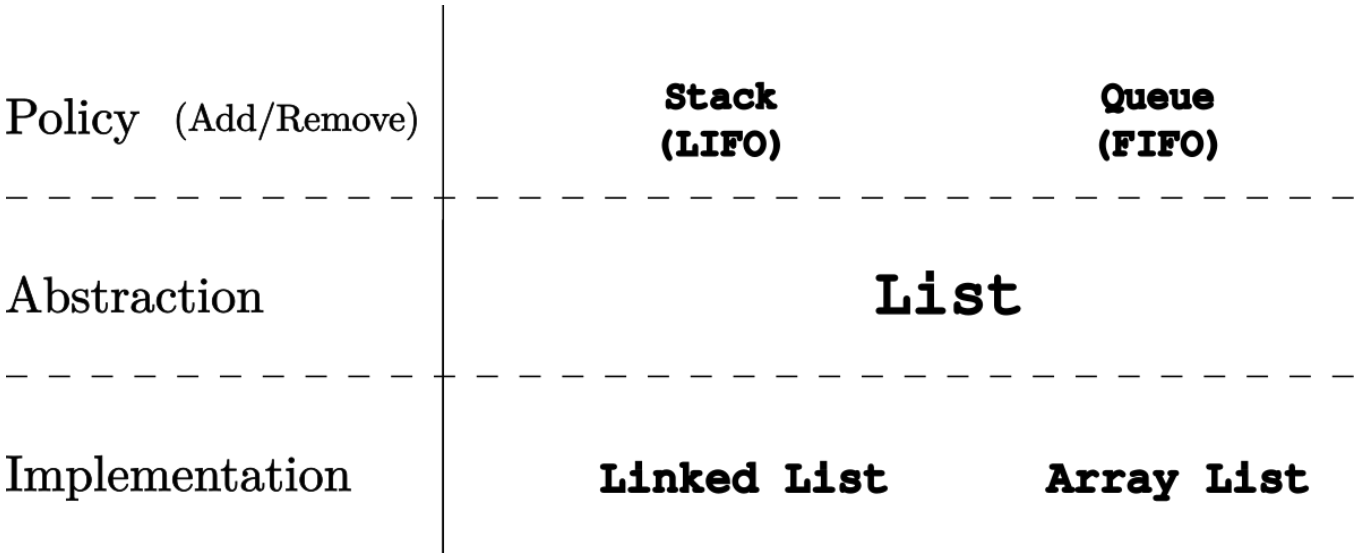
Policy

The word **policy** comes from the Latin *politia* meaning "government" and we generally associate the word to mean "rule". It shares the same root as "police", as in rule enforcement, and "politics", as in rule debate.

A policy in terms of programming is a rule that we will follow rigorously. The problem in programming is that rules are only as strong as the enforcement we create, and it can be easy to sidestep a rule if we do not use robust constraints to force adherence to rules by our programming team and client developers. For example, we can use information hiding to encapsulate and create abstraction layers that support enforcement simply by prohibiting access to the implementation and only allowing access through prescribed interfaces. A policy in programming can take many forms, but one way is to use an abstraction layer called a **policy layer** that not only prescribes the interface for a data type but also specifies the rules that the data type behaviors must always ensure.

As an example, we can specify simple add-remove policies that dictate the behaviors for list interactions. One such policy is that the oldest item added to the list must be the next item to be removed, a policy known in society as first-come-first-served or in the programming domain as **first-in-first-out (FIFO)**. An alternative policy is that the newest item added to the list be the next item to be removed, a policy known in the programming domain as **last-in-first-out (LIFO)**. We encounter these policies in daily life, *e.g.* FIFO is applied in every line you ever stand in and LIFO often emerges when unloading a box or an elevator.

These policies are so regularly encountered at the application level, that programmers have defined formal names and interfaces for lists that follow these two policies. A FIFO list is known as a **queue** and a LIFO list is known as a **stack**.



In other words, a stack is a specialized list that has additional rules that may invalidate the general list interface to ensure the rules are consistently enforced. Similarly a queue is also a specialized list that also may invalidate the list interface for consistent enforcement. Because both are lists, both can be implemented using either an array or a linked-list; however, one implementation may be better suited than the other based on efficiency.

Stack

A **stack** is a specialized list that operates under a last-in, first out (LIFO) insertion and removal policy and we intentionally use language that suggests vertical structuring. For example, we use terminology like top and bottom when referring to the "ends" of a stack and interactions occur exclusively at the top of the stack, *i.e.* a single end of a list. A stack is built on the metaphor of placing items on top of one another and items are typically added to the top of a stack, removed from the top of the stack, and when a stack is empty we have reached the bottom of the stack, so the data structure should conform to these ideas and terminologies.

Remember that abstract data types are really just ideas built on human conceived metaphors, so it is important to use language that conforms to these metaphors so everyone can understand any discussions that involve that data type. Inventing new language for well settled ideas will result in lack of clarity, misunderstanding, and ultimately bugged implementations. For example, novice students have a tendency to use terms like the front of a stack which omits the proper context and raises the question of whether the student means the top or the bottom of the stack.

Stack Interface

The minimal stack interface consists of three interactions:

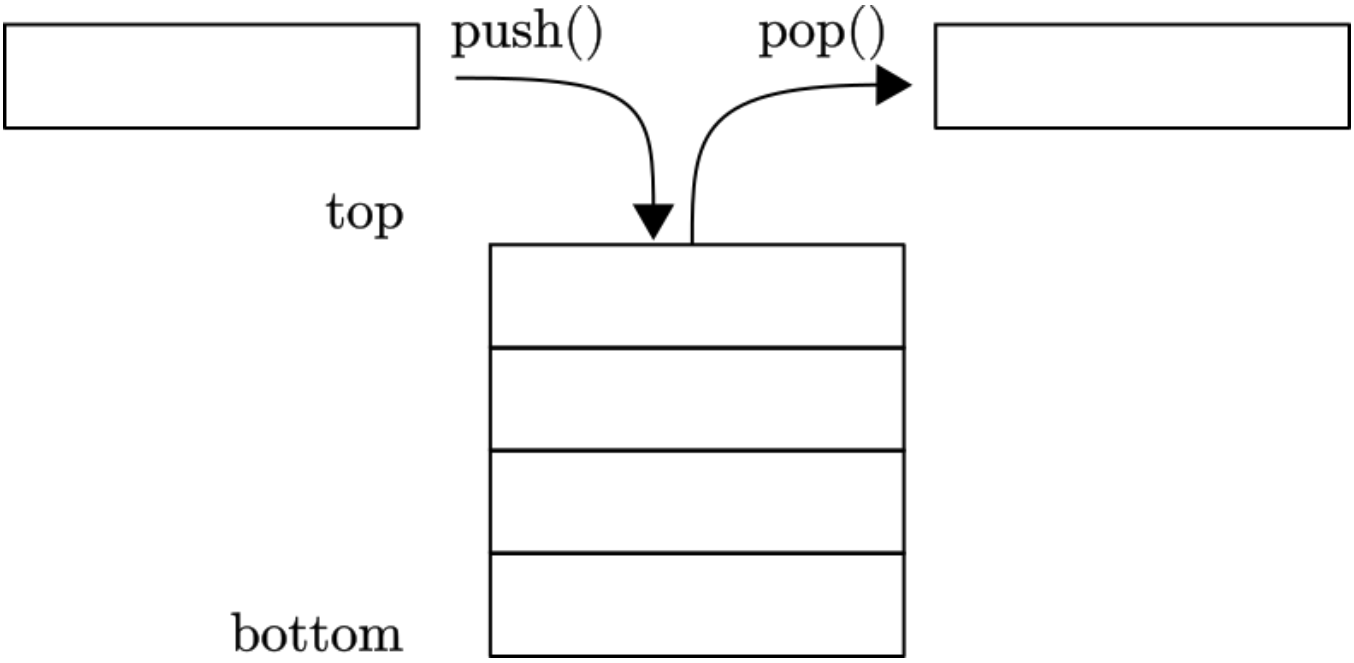
```
void push( element ) : Takes an element as input, places that element on top of the stack, and
returns nothing

element pop( void ) : Takes no input, removes the top element from the Stack and returns that
element

boolean isEmpty( void ) : Takes no input, returns true if the Stack contains no elements;
otherwise, returns false
```

A fourth interaction is also common for stacks but it is not a necessity for basic stacks:

```
element peek( void ) : Takes no input, returns the top element from the stack without removing
the element from the stack, *i.e.* the stack must remain unchanged.
```

> **Feature Creep, Trust, and Developmental Efficiency**
>
> It may seem like that the stack interface is too simple because it does not include interactions like `remove(element)` or `get(index)`, and as a result you may be inclined to add more and more "features", *i.e.* interactions, to the interface and there is no doubt that some features may be welcome additions; however, this can lead to "feature creep" and may start to blur the lines between different data structures and worse may allow class users to violate the policy that the data structure is supposed to enforce. Before adding more features to a stack consider that a stack has very specific rules it MUST enforce, *i.e.* LIFO insertion and

> removal, so methods that may violate this policy should generally not be added to the stack interface. A stack with an additional `remove(element)` method is no longer a stack and is instead some form of hybrid data structure that supports both list and stack features and should not be extended much trust in policy enforcement on the part of the class user. Efficiency is determined at the implementation level but we are intentionally hiding the implementation. A feature like `get(index)` can be inefficient in a linked-list, so we don't really want to expose it in the stack just to allow the stack to be used like a list. If system users want to use a stack like an array list, make them create an array list instead and give them interactions that allows conversion back and forth between array list and stack. It is impossible to write one data structure that does everything efficiently and when you try to do so it starts to do everything poorly, so create data structures that fulfill specific roles extremely well and force users to use the data structure correctly by controlling interactions with the data structure.



A stack is empty if the top of the stack is at the bottom and a stack may have a maximum capacity determined when the stack is initially created, especially if the stack uses an array-based implementation.

Stack Applications

We have talked about one specific stack application throughout the semester and that is the memory stack. The stack in the memory model operates exactly like a stack: we push a new stack frame onto the stack every time we call a function, we pop the current stack frame (top) from the stack when a function returns. We call it "The Stack" because it is a stack.

In the real world, we see stack behaviors naturally emerge in situations like the loading and unloading of an nearly full elevator. A new passenger squeezes into the elevator and stands at the front because there is no room to maneuver inside the elevator. At the next stop, this new arrival must get off the elevator before anyone else can get off. The full elevator is the correct example here because in a non-full elevator, humans will self reorganize inside the elevator car because they don't want to have to exit the elevator car before reaching their destination.

In a more general sense, stacks are incredibly useful in maintaining a history of data and facilitate going back to an earlier step in that history. This is one way to implement the back button on your browser or the undo (Ctrl-Z) option in most applications. In these applications, a stack can be used to record the last x actions and you can step back any number of actions less than x by popping the history stack. In a browser, it back and forward can be modeled using two stacks, one for back and one for forward, such that when back is activated, the topmost link on the back stack is popped and that link is then pushed onto the forward stack and vice versa when the forward button is activated.

Queue

A **queue** is a specialized list that operates under a first-in, first out (FIFO) insertion and removal policy and we intentionally use language that suggests horizontal structuring. For example, we use terminology like front and back when referring to the "ends" of a queue and interactions occur at both ends of the queue, *i.e.* at both ends of the list. A queue is built on the metaphor of standing in line and people typically join the back of a line and leave when they reach the front of a line, so the data structure should conform to these ideas and terminologies.

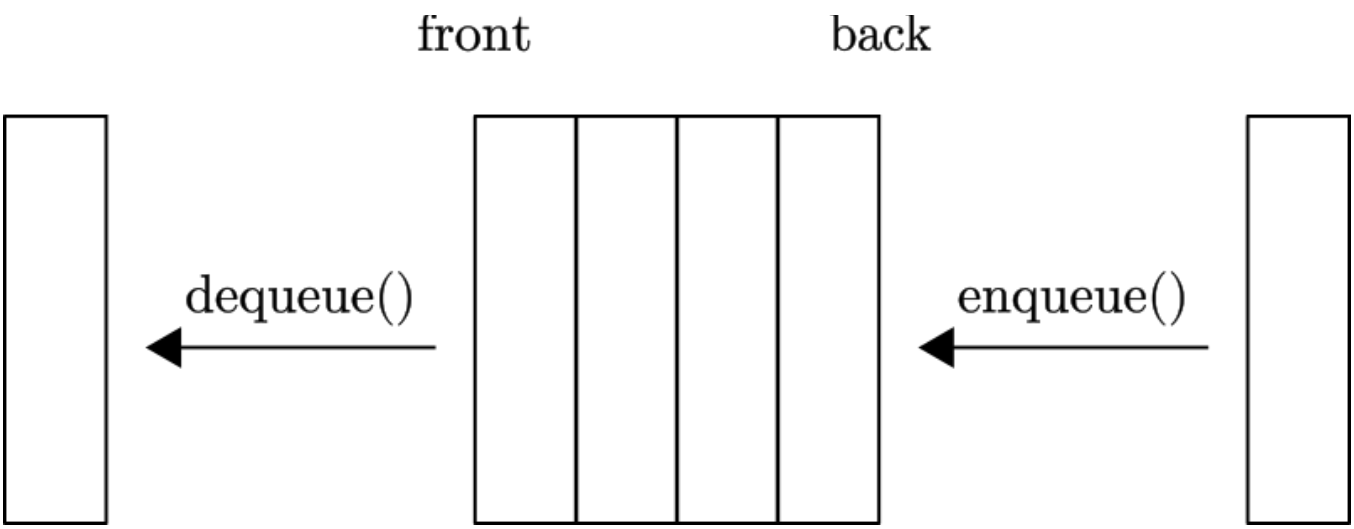
Queue Interface

The minimal stack interface consists of three interactions:

```
void enqueue( element ) : Takes an element as input, places that element at the back of the
Queue, and returns nothing

element dequeue( void ) : Takes no input, removes the front element from the Queue and returns
that element

boolean isEmpty( void ) : Takes no input, returns true if the Queue contains no elements;
otherwise, returns false
```

Similar to stacks, the peek interaction is also common for queues but it is not a necessity for basic queues:

```
element peek( void ) : Takes no input, returns the front element from the queue without
removing the element from the queue, *i.e.* the queue must remain unchanged.
```



Most queues can be considered empty if the front of the queue is at the back and a queue may have a maximum capacity determined when the queue is initially created, especially if the queue uses an array-based implementation.

Queue Applications

We encounter queues on a daily basis in society because we structure many interactions around waiting in line, so we will need a queue in an application whenever we need to model the idea of standing in line. At a high level, queues are used to model play-lists in audio and video streaming services, for waiting models like buying concert tickets on a virtual marketplace or waiting to be matched in video games, or for other applications.

There are more sophisticated forms of queues where there are multiple lines where certain lines have specific priority. For example, some airlines have "priority boarding" and they model this by assigning passengers priority groups and many airports have priority security where passengers who have been "prescreened" are allowed to us a security lane dedicated exclusively for qualified passengers.

Implementation

Recall that the stack and queue are just lists with specific rules for adding and removing elements and the basic stack implementation only requires we support interactions at one end

Stack Implementations

When interacting with a stack, we will interact with only one end of a list data structure, so we want to implement our interactions such that they use the underlying data structure in the most efficient manner. For example, adding and removing at the front of an array is always $O(n)$, but adding and removing at the back of an array can be $O(c)$ if there is space in the array.

Remember that worst-case does not mean intentionally creating a bad design for data structure. Worst-case naturally emerges from usage in an application as a function of data in the data structure which can be difficult to detect and control. We intentionally design our algorithms and data structures, so we can make choices that may mitigate worst case scenarios.

Linked List Based Stack

If we were to implement a stack using a linked list, what design elements are needed to support the fastest insertion and removal policies? Considering that a stack requires interactions at only one end and all linked-lists have a head, we should ask whether or not push and pop, *i.e.* add and remove, can be done in constant time at the linked-list head.

Recall that adding to the head involves the following code:

```
Node n = new Node(data);
n.next = head;
head = n;
```

The number of lines to add to the head does not vary with the size of the list, so adding at the head can be done in constant time.

Recall that removing from the head involves the following code:

```
ListNode it = head;
head = head.next;
return it.data;
```

The number of lines to remove from the head does not vary with the size of the list, so removing at the head can also be done in constant time.

Note that we are still retaining the head terminology introduced in linked lists for consistency, but we could easily instead call it top to conform to stack terminology. In other words, in a linked list implementation, the head acts as the top of the stack.

Since both add and remove can be implemented in our most basic singly linked list with a head pointer only in constant time, we can easily support push and pop with constant time operations using a linked list implementation. We can also test whether or not the stack is empty in constant time by a simple test of whether or not the head is null and we don't really need to keep track of counts or other features to support the basic stack with a linked list.

This suggests that a linked list is an excellent data structure to support stacks in terms of time complexity. However, recall that each element in the basic linked list requires two pieces of information to maintain each element in the list: the data and the next pointer. It is worth considering whether the cost in terms of space is too expensive and an alternative implementation should be sought.

Array Based Stack

If we were to implement a stack using an array, what design elements are needed to support the fastest insertion and removal policies? The challenge we face with an array based stack is that stacks grow and shrink dynamically while arrays are more static and for an array to grow and shrink, it requires relatively costly, *i.e.* $O(n)$, copying because we have to allocate a whole new array and copy data from the old array into the new array in order to "grow" the array. Is it possible to mitigate this weakness somewhat so that we do not need to copy arrays at all or at the very least copy with very low frequency.

Inserting and removing elements at the low order index of the array is going to result in $O(n)$ efficiency for insertion and removal. If we force insertion to occur strictly at the 0 index, we have to make space for the new element which would require us to shift all elements to the right assuming there is space in the array. If we force removal to occur strictly at the 0 index, we have to eliminate the space created when the element is removed which would require us to shift all elements to the left.

Suppose instead we have an array that has more space allocated than it currently uses and we insert and remove elements at the smallest available high order index (in a zero based indexed array, this is the index `n` which is the `n+1` position and element in an array since it is zero based), what is the efficiency of insert and remove?

In this array, the n index is a legal position because the space is already allocated and because arrays support constant time access to any element in the array, we can assign the element to the `n+1` position in constant time.

```
int[] A = new int[2 * n];  // allocate more space than needed
count = n;                 // array contains n elements
A[n] = value;              // inserting a new element in O(c)
count = count + 1;         // updating the number of elements
```

Similarly, in this array we can remove an element from the highest order position in constant time because we do not need to allocate a new array (just absorb the space into the array's **free space**, *i.e.* the difference between the array's **used space** and array's **allocated space**) and copy when an element is removed. In fact, we don't need to technically remove the element, we just need to reduce the count of elements.

```
count = count - 1;         // removes the last element
```

Based on this argument, assuming we have sufficient free space in an array, we can support constant time insertion and removal if we perform these operations at the highest order index in the array's used space. This does raise several questions: how much space we should provide for an array that needs free space, what we should do when the array is full, and are there other ways to use arrays that will allow us to operate on both ends of the data without expensive shifting.

Trading Space Efficiency for Time Efficiency Memory is cheap and plentiful. While improvements in processor speeds has reached a number of physical limits that raise questions about future processor speeds in terms of Moore's law, memory has consistently grown according to Moore's Law which has resulted in extremely large amounts of memory for very low prices in all forms of electronics on the current market. Arrays have a very tight relationship between space and time performance especially if we only allocate just enough space for the array to hold existing data. However, if we allocate more space that we anticipate for an array, we can minimize the number of times we have to grow an array. When we grow an array, the naive approach is to allocate an array of only one additional element which we will immediately fill up forcing subsequent insertions into the array to grow again. This leads to $O(n)$ time complexity for all inserts (best case, average case, and worst case) and $O(n)$ space complexity overall for the array. Is there an alternative that can improve the efficiency? Consider that if we fill up an array of size n and have to grow it in anticipation of further usage, it is a reasonable assumption that since we needed n spaces to hold the existing data, we will need an additional n spaces for future growth. In other words, when we grow an array, we double the size of the previous array in the new allocation. This suggests a simple policy that helps minimize the frequency of the growth. In terms of efficiency, this strategy is still $O(n)$ time complexity for insert in the worst case of the array being full and $O(n)$ space complexity overall for the array;

however, the best case and average case time complexity fall to $O(c)$ and the frequency of the worst case is minimized to occur on a logarithmic time scale.

Queue Implementations

A queue is similar to a stack but different enough that the implementation needed to support a queue will require additional features for the underlying data structures. Recall that interacting with a queue requires interactions at both ends of the list data structure, so again our implementations need to support the interactions such that they produce the most time efficient behaviors.

Linked List Based Queue

A queue requires efficient insertion at one and efficient removal at the other end. This raises the question of whether or not a linked list can support constant time insertion at one end and constant time removal at the other end and what features of the linked-list are needed to support these behaviors. Consider the following questions:

- Can we insert and/or remove at the head in constant time?
- Can we insert and/or remove at the tail in constant time?
- Does the linked-list need a tail pointer?
- Does the linked-list need to be singly-linked or doubly-linked?
- What is the design that minimizes space efficiency?

In the discussion of linked-list based stacks, we demonstrated that any linked-list with a head pointer supports constant time insertion and removal at the head.

Constant Time Insertion at Head

```
Node n = new Node(data);
n.next = head;
head = n;
```

Constant Time Removal from Head

```
ListNode it = head;
head = head.next;
return it.data;
```

We can insert at the tail once we locate it:

Insertion at Tail

```
it.next = new Node(data);
```

But we can eliminate the cost of finding the tail if we include a tail pointer

Constant Time Insertion at Tail

```
tail.next = new Node(data);
tail = tail.next;
```

However to remove from the tail, we need to find the element preceding the tail which requires an $O(n)$ iterative search through the linked-list if the linked list is singly linked. However, we could doubly-link the linked-list which would give us constant time access to the element preceding the tail.

Constant Time Removal at Tail for Doubly Linked-List

```
tail = tail.prev;
tail.next = null;
```

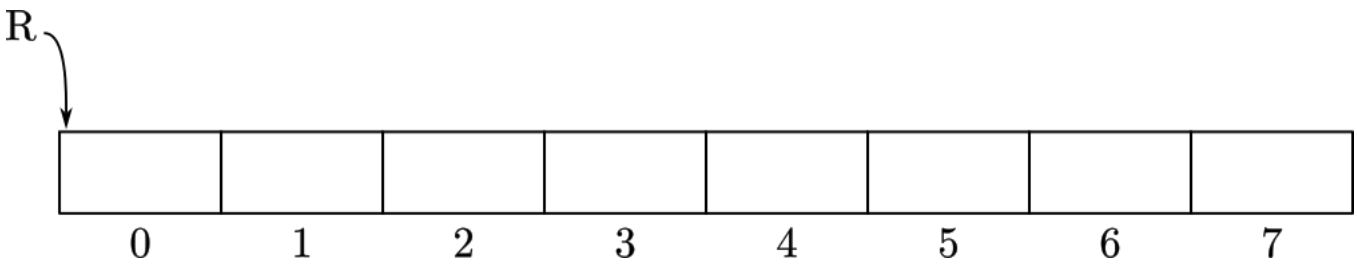
But this does now require a doubly linked-list just to support an operation we don't really need. Consider that the singly linked list with a head and tail meets all of our requirements, *i.e.* constant time insertion at one end (tail) and constant time removal at the other end (head). Using a doubly linke,d-list does not increase the space complexity but it does double the number of references per element unnecessarily for no additional benefit. In some applications, space is still a premium, *e.g.* operating systems and low-level device programming, so it is important to be able to support data structures with the minimum implementation that services all requirements with the best time complexity.

Array Based Queue

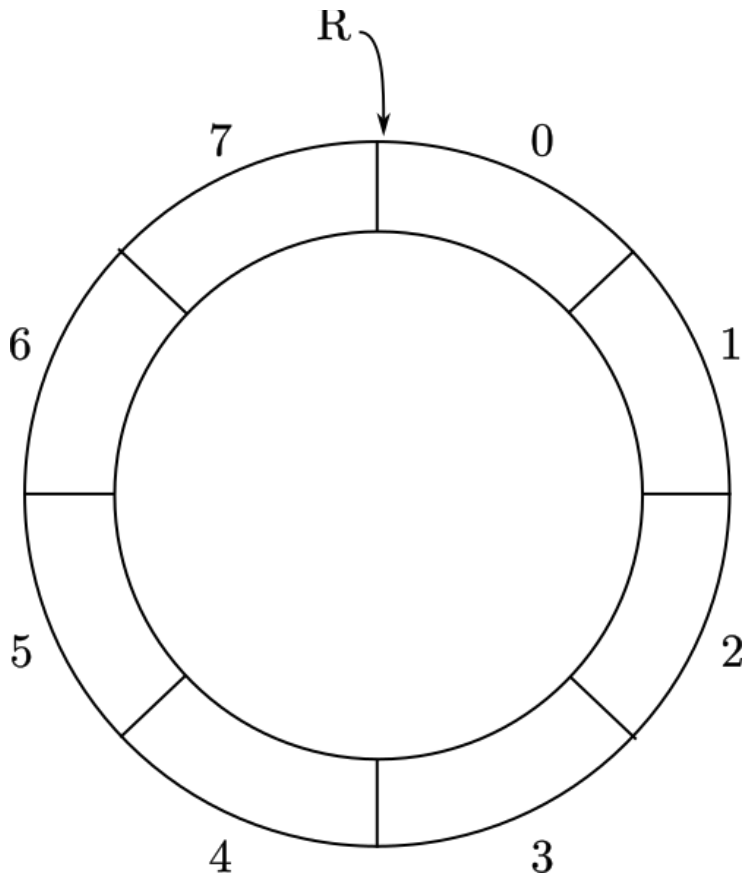
On the surface, using an array for a data structure that requires data to be inserted or removed from the low order indices is inefficient if we take a conventional approach with an array where we shift data around. However, shifting data is not the only approach we could take. Instead we might shift around the indices demarcating the front and back of the queue. In other words, instead of moving the data around, move the pointers to the head and tail around. In the conventional approach, we assume the front of the queue is at index 0 and the back of the queue is at index `n-1`.

Ring

The **ring** data structure is a queue implemented with an array that uses multiple indices and simple but clever math for index incrementing. Let `R` reference an array that will be treated as a ring.



The challenge is how can we use a linear data structure in such a way that it behaves like a circular structure using very simple logic to ensure high reliability.

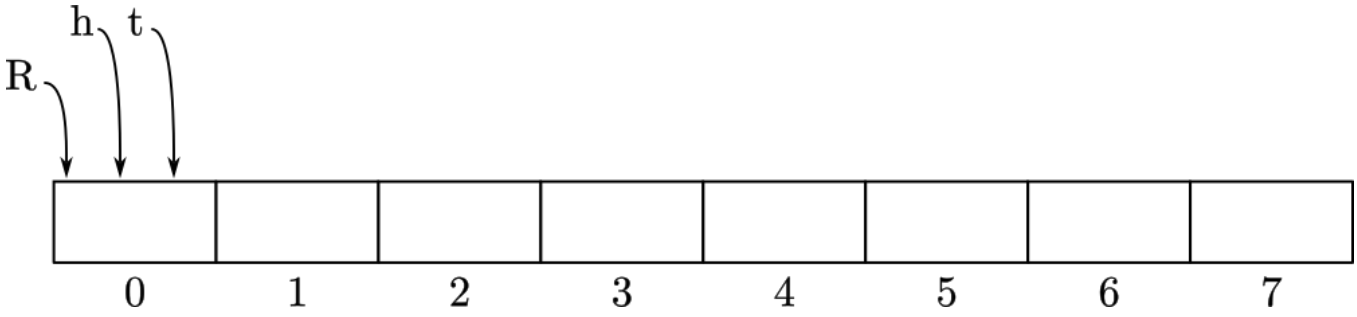


As with any queue we will need data elements that track the head and tail of data in the ring; however, since this is an array, we can track the head and tail elements using integer values instead of using reference types. The effect is similar (and raises some interesting questions about what reference types really are) and allows integer math to be performed on the indices. Let `h` be the index of the head and `t` be the index of the tail.

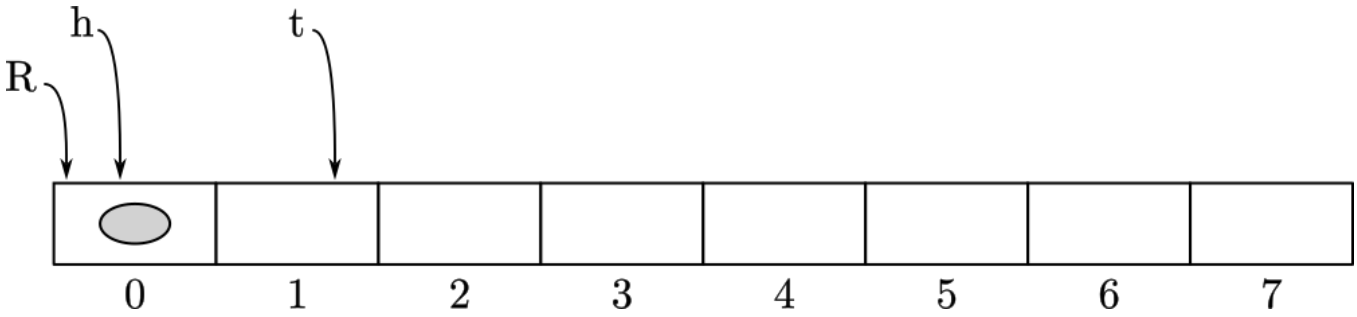
```
class Ring {
    private int h;
    private int t;
    private int[] data;
    private int count;

    public Ring( int size ) {
        data = new int[size];
        h = 0;
        t = 0;
        count = 0;
    }
    ...
}
```

In Java, `h` and `t` would not be reference types and instead are strictly defined as `int` types. However, they technically are acting as pointers, so we will use pointer representations in the following illustrations. This does make sense because a reference is a form of index as an address is just a memory index.

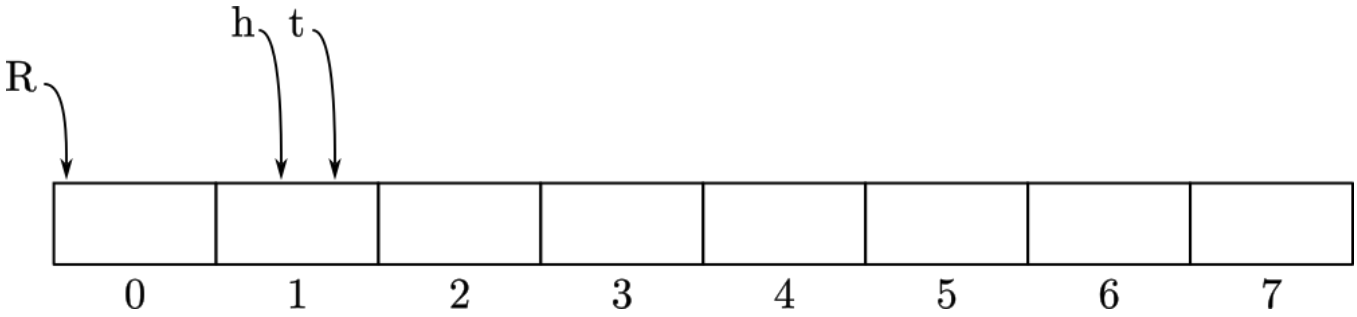
Given the code above, our ring is initialized as the following.



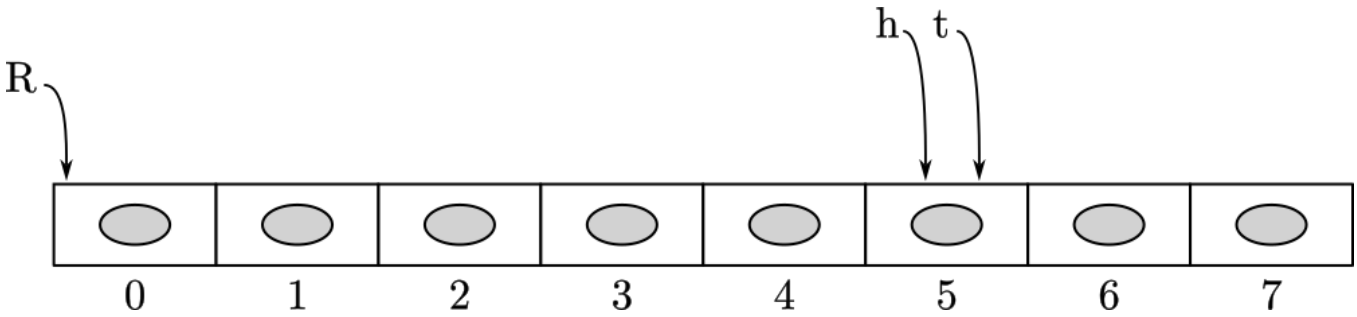
When we enqueue an element to the ring, we insert it at the position `t` and then advance `t`.



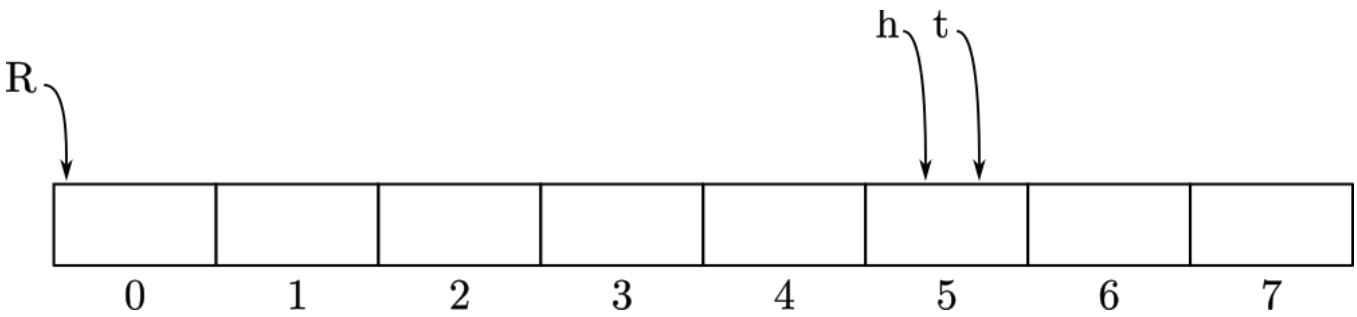
When we dequeue an element to the ring, we remove it from position `h` and then advance `h`.



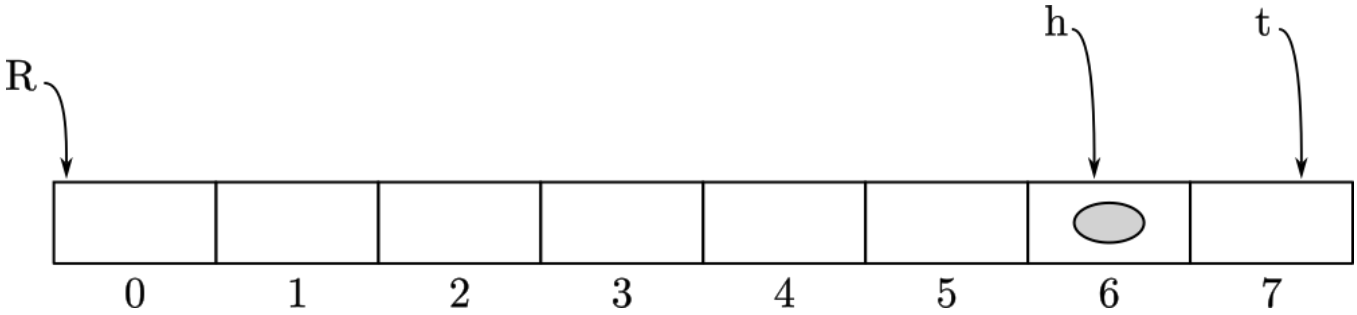
If a ring is full, `h` and `t` are equal and the count is non-zero.



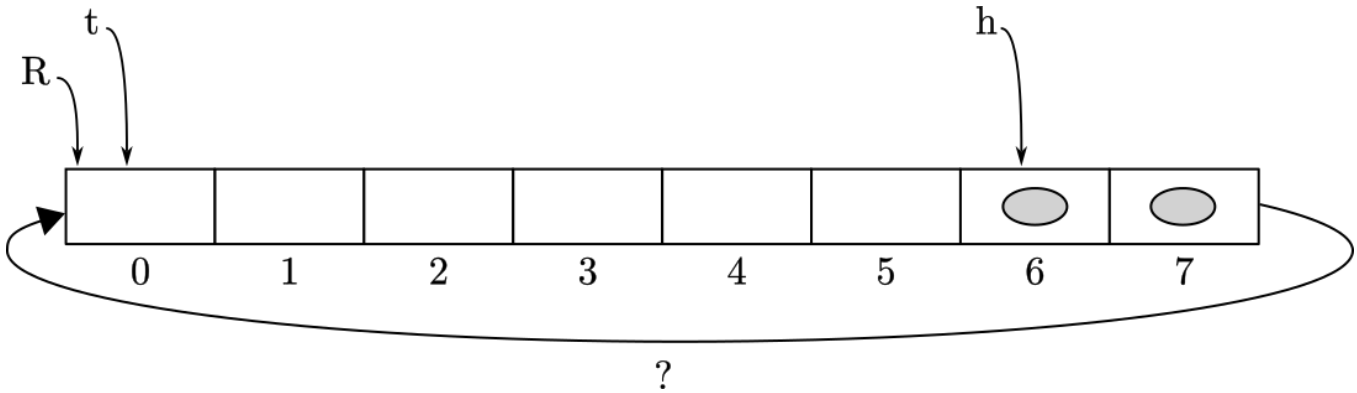
If a ring is empty, `h` and `t` are equal and the count is zero.



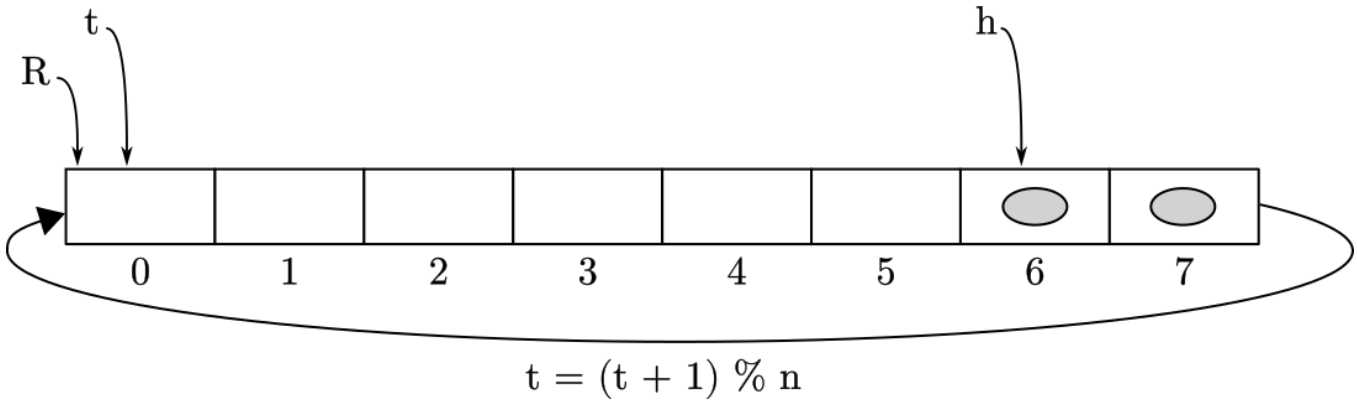
If one of our indices contains the index of the last element in the array, how can we advance it? For example, `t` equals $n - 1$ and advancing it by simply adding one will cause it to index beyond the legal boundary of the array



So how do we advance `t` (or `h`) in such a way that it contains the value of the beginning index of the array?



The first inclination is to write conditional logic that tries to address this issue; however, this logic can become surprisingly complex if the data structure has more complex features like allowing multiple items to be enqueued simultaneously. Can we instead handle this mathematically and robustly such that we can completely eliminate any if statements? Yes, if we advance an index using the modulo operator (`%`).



Recall that the modulo operator computes the remainder of integer division.

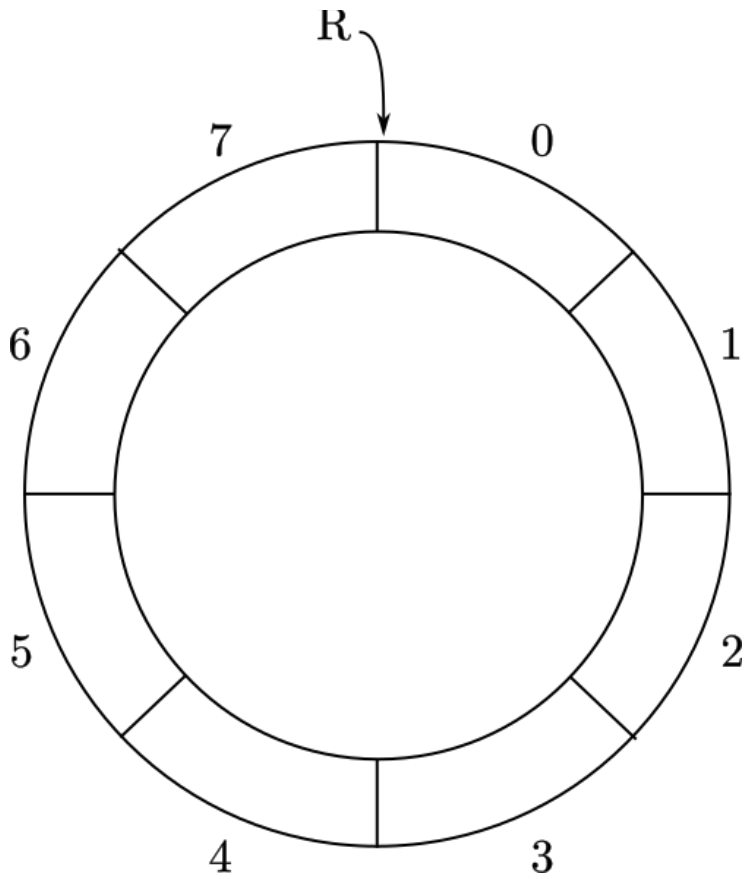
The equation:

$$i = (i + 1) \mod n$$

ensures that when adding to a value that should remain bounded in the range $[0, n - 1]$, the value remains in the range. Consider the following:

$$\begin{aligned}
&0 \mod n = 0 \\
&1 \mod n = 1 \\
&2 \mod n = 2 \\
&\cdots \\
&(n - 2) \mod n = n - 2 \\
&(n - 1) \mod n = n - 1 \\
&(n - 0) \mod n = 0 \\
&(n + 1) \mod n = 1 \\
&\cdots \\
&(2n - 1) \mod n = n - 1 \\
&(2n - 0) \mod n = 0 \\
&(2n + 1) \mod n = 1 \\
&\cdots \\
&(cn - 1) \mod n = n - 1 \\
&(cn - 0) \mod n = 0 \\
&(cn + 1) \mod n = 1
\end{aligned}$$

In all cases, including the last few examples which demonstrate arbitrary values around the range boundary, the math just works out such that the result of the calculation is always within the range. In other words, the use of modulo allows creation of a simple math expression that automatically resets the calculation such that it can never violate the range and the index will never be out of bounds. This eliminates the need for any if statements and reduces the likelihood of introducing a bug in the indexing logic. It also conceptually wraps the indexing of an array into a circle, *i.e.* indexing goes around and around without limit on the number of cycles but still bounded in the legal range, and practically the array behaves like a circular data structure, hence the name ring.



Double-Ended Queue (Deque)

The **Deque** or **Double-Ended Queue** is a generalization of the queue into a data structure allowing for add and removal at both ends of the queue and therefore it is capable of acting both as a stack producing LIFO behavior and as a queue producing FIFO behavior. Due to the increase in complexity and the hybrid nature of the structure, there is no standard interface for a deque so method labels should need to expand to clearly indicate the location of the add or the remove operation. For example, an deque interface might include the following methods among others:

```
addFront(element)
addBack(element)
element removeFront()
element removeBack()
```

With the added requirements, it is necessary to use a more complex implementation to support it, so a deque will generally be implemented using a doubly-linked list including both `head` and `tail` pointers so that it can support the requirements in constant time. Because the deque must support efficient operation of both add and remove at both ends, an array is typically a poor choice for the implementation; however, if it is possible to fix the size of the structure at the beginning, a deque could be implemented using the same strategies used with the Ring to produce a constant time array-based deque.

The deque is the more appropriate solution to the browser back button and browser forward button example because the back and forth behavior can be managed in the deque alone and eliminates any need for a second data structure.

Review

1. Explain the First-In, First Out policy.
2. Explain the Last-In, First Out policy.
3. Define the standard minimum interface for a stack
4. Define the standard minimum interface for a queue
5. Define the insertion/removal policy for a stack.
6. Define the insertion/removal policy for a queue.
7. To have stack behavior emerge, how many ends of a list do you interact with?
8. To have queue behavior emerge, how many ends of a list do you interact with?
9. Describe the full implementation of a stack using an array.
10. Analyze the efficiency of stack operations when implemented using an array.
11. Describe the full implementation of a stack using a linked-list.
12. Analyze the efficiency of stack operations when implemented using a linked-list.
13. Describe the full implementation of a queue using an array.
14. Analyze the efficiency of queue operations when implemented using an array.
15. Describe the full implementation of a queue using a linked-list.
16. Analyze the efficiency of queue operations when implemented using a linked-list.
17. Give an example of stack behaviors in the real world.
18. Give an example of a software application where the stack is the best suited data structure.
19. Give an example of queue behaviors in the real world.
20. Give an example of a software application where the queue is the best suited data structure.